# Research on an embedded-based mask wearing detection system

Yong Chen
School of Computer and Information Technology
Hefei University of Technology
Hefei, China
e-mail : ch1515696@qq.com

2023 4th International Conference on Computer Vision, Image and Deep Learning (CVIDL) | 979-8-3503-2644-4/23/$31.00 ©2023 IEEE | DOI: 10.1109/CVIDL58838.2023.10166278

***Abstract*—Since the outbreak of the new crown epidemic began worldwide, the virus has been spreading very fast, and the most convenient and effective way to deal with the spread of the virus is to wear a mask properly. Up to now, the new crown is still in epidemic, not only the new crown virus, but also the current outbreak of influenza A virus. In order to reduce the speed of virus spread and protect our own health, in some public places, we have to wear masks according to the epidemic prevention and control requirements. In this paper, we propose a mask wearing detection system based on embedded Linux, using the optimized and improved YOLOv5 network model, and deploy it to the embedded system to achieve real-time detection of mask wearing.**

***Keywords- Embedded systems; Linux; Mask wearing detection; YOLOv5***

## I. INTRODUCTION

Since 2020, the New Crown epidemic has seen wave after wave of outbreaks of different scales, bringing heavy damage to people all over the world. Tens of millions of people have died worldwide because of the New Crown virus, and many national and regional healthcare systems have been overwhelmed by the surge of New Crown cases. The New Crown epidemic has taught us the importance of wearing masks, not just to prevent the spread of the New Crown virus, but also other viruses or bacterial classes, such as the SARS virus in 2003, the bird flu virus that followed, and the influenza A virus that is currently breaking out in many parts of the country.

Currently, intensive and extensive research in deep learning is rapidly advancing artificial intelligence and machine learning. In the field of computer vision, deep learning has shown very superior results and has made great progress in many areas. The mask wearing detection system in this paper is in the domain of computer vision. Deep learning first emerged in the 1980s. The first to emerge was artificial neural networks, which were the first network model for deep learning, but the single-layer perceptron in neural networks was limited to linear problems only and could not be classified for non-linear problems[1]. Later, Rumelhart Hinton et al. proposed the backpropagation algorithm, which has since enabled neural networks to solve linear problems and made the training of MLPs (multilayer perceptrons) feasible[2]. However, some drawbacks of the backpropagation algorithm have been revealed, namely that the training speed of the model is inversely proportional to the number of neuron nodes. This causes the study of neural networks to be limited by hardware conditions. This drawback of non-convex optimisation makes the shaved descent method often makes the network eventually stagnate at a local optimum solution. In addition, as the number of layers in the network increases, it may make the gradient's disappear and it will not improve the learning ability of the network to get a good parametric model. The development of neural networks has not had a fast development for several years because of these problems.

Target detection techniques have been evolving, initially using manually designed features for classification and later introducing convolutional neural networks to extract target features, there is no need for manual feature extraction. However, the target localization problem mainly uses sliding windows, leading to an increase in the window size as the number of pixels increases and the need to search multiple scales. Therefore, an efficient detection framework becomes crucial. Two main classes of target detection algorithms exist: two-stage target detection algorithms and single-stage target detection algorithms. The two-stage algorithm divides the detection step into two steps, first extracting the candidate region features, and then using a detection network for detection and classification. However, this algorithm requires higher computational resources and has higher complexity. For example, Girshick's R-CNN[3] algorithm performs feature extraction by extracting a large number of candidate regions from an image and then uses an SVM classifier[4] for classification, but this method takes relatively long time for detection. For this reason, improved algorithms have been continuously researched, such as SPPnet[5], Fast R-CNN[6], Faster R-CNN[7], R-FCN[8], SSD[9], YOLO series[10], etc. Among them, SSD, YOLO belong to single-stage target detection algorithm, which can achieve end-to-end operation without first selecting the candidate region, and can directly generate its class as well as its location information, and this single-stage detection algorithm has the advantage of fast detection speed.

Mask wearing detection is a target detection task, where for a given picture as input, classification is performed by locating the face and the mask to be able to determine whether the target is wearing a mask or not[11]. Nowadays, there are not many detection algorithms that are specifically designed to detect masks, but deep learning is moving forward every day and it is relatively easy to detect mask wear with common target detection algorithms.The improved self-attention mechanism is introduced into the feature pyramid network, which greatly increases the expressiveness of the extracted feature maps.



Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 06:38:59 UTC from IEEE Xplore. Restrictions apply.

However, there are still cases of wrong detection and missed detection, and the detection efficiency is not high[12].

In this paper, the design and construction of embedded Linux hardware platform and software structure are completed by using I.MX6ULL chip to address the problems of high cost and high power consumption of existing embedded platform based on deep learning. The Linux kernel is studied, the porting of the system boot program U-Boot, the cutting and porting of the Linux kernel, the construction and porting of the root file system, the compression and improvement of the network model for the problem that the deep learning network model is relatively large, the number of parameters of the model is reduced, the detection speed of the system is improved without losing accuracy, which makes it easier to implement the deep learning algorithm in the embedded system. The deep learning algorithm is implemented in the embedded system, and finally realizes the real-time detection of whether the mask is worn or not.

## II. MATERIALS AND METHODS

### A. Build embedded Linux platform

#### 1) Installing the cross-compiler

The main reason for cross-compiling is to develop and build software on one architecture and run that software on a different architecture. This is typically a task performed on embedded systems, mobile devices, and other specific devices. With cross-compilation, binaries for multiple different platforms can be built on a single platform, which saves time and reduces errors because there is no need to build and test the software separately on each target platform. Cross-compiling facilitates cross-platform development, allowing developers to more easily share and reuse code between different platforms. This is useful for developing applications with high portability, as the code can be easily ported to new platforms. To perform cross-compilation, we need to install the corresponding cross-compilation tool chain on the host platform. Since our embedded system is of ARM architecture, we need to install the arm-linux-gcc cross-compiler on our Linux PC, which can compile executable code for the Linux ARM platform.

#### 2) Porting the bootloader U-Boot

The bootloader is one of the most important components of a computer system. Its main function is to initialize hardware devices and load the operating system kernel when the computer boots. Usually, the bootloader is stored in the first sector of the boot device (e.g. hard disk, floppy disk, USB drive), which is called the master boot record (MBR). When the computer starts up, the bootloader is executed first. Then, the bootloader reads the first sector (usually 512 bytes) from each boot device according to the predefined boot sequence and loads it into the computer memory for execution. Download U-Boot from the official website, unzip it, change ARCH=arm and CROSS_COMPILE=arm-linux-gnueabihf- to specify the cross-compiler to use, and finally compile U-Boot and burn the resulting u-boot.bin file to memory.

#### 3) Porting the Linux kernel

As with U-Boot, the Linux kernel has to be configured before compiling the Linux kernel. Each board has its own default configuration file, which is stored in the arch/arm/configs directory. After the Linux kernel is compiled, the zImage image file is generated in the arch/arm/boot directory, and if we need to use the device tree, the .dtb (device tree) file for the development board is generated in the arch/arm/boot/dts directory. . After the compilation is complete, use the tool to burn the generated zImage file to the development board.

#### 4) Creating and porting the Linux root filesystem

The root file system is an important part of the Linux system, it contains all the files and directories of the Linux system and is the foundation of the whole system. We use BusyBox to build the root file system. After downloading the source code, we configure the relevant parameters to configure the root file system and burn the file system into the embedded device after the production is finished.

### B. Mask wearing detection based on deep learnning

#### 1) Theory of target detection

Object Detection is one of the core problems in the field of computer vision, where the task is to determine the class and location of all objects of interest in an image. Object detection has always been one of the most challenging problems in computer vision, despite the different appearance, shape and pose of various objects, as well as the interference of lighting and occlusion during imaging. In the field of computer vision, image recognition usually consists of four main types of tasks: firstly, classification, which solves the "what" problem, i.e. given a picture or a video, determine what category of target it contains; secondly, localisation, which solves the "where The second is location, which solves the "where" problem, i.e. locating the specific location of the target. The third is detection, which solves the "where is what" problem, i.e. locating the location of the target and determining what the target is. The fourth is segmentation, divided into Instance-level and Scene-level segmentation, which solves the problem of "which target or scene each pixel belongs to".

Deep learning-based target detection algorithms are divided into two main categories: Two stage and One stage. The deep learning model used in this paper is YOLOV5[13], which is a typical One stage. One stage does not require region generation first and extracts features directly in the network to predict object classification and location. The One-stage algorithm usually has the following steps:

Step 1: Feature extraction: The original image is fed into a deep neural network for feature extraction, generally using a convolutional neural network (CNN) to extract image features.

Step 2: Candidate frame generation and classification: For each pixel point on the feature map, a series of candidate frames of different sizes and aspect ratios are generated and each candidate frame is classified to determine whether it contains the target object.

Step 3: Candidate frame regression: For candidate frames classified as targets, their bounding box coordinates are fine-tuned to locate the target more accurately.



Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 06:38:59 UTC from IEEE Xplore. Restrictions apply.

Step 4: Non-maximum suppression (NMS): the NMS algorithm is used to filter overlapping candidate frames to obtain the final target detection results.

*2) Dataset*

The datasets needed in this project are from the network, including 7900 ALZOO datasets, 3000 RMFD datasets, and 100 frames from some videos.It is used for training, validation, and testing datasets. After collecting enough datasets, these images are divided into two categories, one is mask and the other is no_mask, and then we need to label the datasets, we use labelImg for labeling. After the labeling is completed, the dataset is organized into training set, validation set and test set, with the ratio of 50%, 10% and 40% respectively.

WIDER FACE its a dataset for face detection maintained by the laboratory of the Chinese University of Hong Kong. For each event category, 40%/10%/50% of the data is randomly selected as the training set, validation set and test set. MAFA (Masked FAces) is a masked face detection dataset open sourced by the Chinese Academy of Sciences, which has various images of masked faces, most of which are masked faces. faces in this dataset have various orientations and masking degrees, and AIZOO has open sourced nearly 8000 annotated images of faces from the WIDER Face and MAFA datasets. Software Engineering Technology Research Centre, Wuhan University. This dataset must contain multiple images of masked and unmasked faces of the same person.

*3) Compression and optimization based on YOLOv5 model*

This paper is based on practical applications to deploy lightweight target detection models on embedded devices, then it is necessary to solve the model size problem. Currently for model compression, the mainstream approaches are structured pruning and unstructured pruning, both with their own advantages and disadvantages. Structured pruning leads to a significant decrease in the accuracy of the model, and unstructured pruning causes a decrease in the inference speed of the model. In this paper, we adopt a chunked weight pruning method to improve the accuracy while giving full play to the hardware capability and improving the model inference speed. The specific process is to divide the weight matrix of each layer into multiple blocks of equal size, and each block contains n consecutive channels of m consecutive filters. The weights of all filter channels at the same position are trimmed in the same block, and the position of weight trimming is flexible and variable between different blocks. It has the flexibility of unstructured trimming, which ensures high accuracy, and also retains the advantages of structured trimming, which can speed up inference.

### *C. Deploying deep learning models to embedded systems*

After the deep learning model has been trained, the model is compressed. As the trained network model cannot be run directly on an embedded system. Generally, we need to convert the model to an inference framework. Open Neural Network Exchange (ONNX) is a set of open formats for representing deep neural network models. ONNX defines a set of environment- and platform-independent standard formats that provide the basis for interoperability of AI models, allowing them to be used interactively across frameworks and environments. Hardware and software vendors can optimize model performance based on the ONNX standard to the benefit of all frameworks compatible with the ONNX standard - in short, ONNX is the middleman for model transformation. Whatever training framework we use to train models, we can all convert those models to ONNX format for storage. Once the network model data is available in ONNX format, the appropriate inference framework can be selected for inference. This paper uses the ncnn inference framework, a high-performance neural network forward computation framework optimized for mobile phones. ncnn was designed from the ground up with mobile deployment and use in mind, with no third-party dependencies, cross-platform, and a mobile CPU that is faster than all known open source frameworks. Based on ncnn, developers can easily port deep learning algorithms to mobile phones for efficient execution. After cross-compiling the ncnn library, the ONNX-formatted network model file is converted into an ncnn-formatted file. Once this is done, the application can be written to call the deep learning algorithm. The input to the model is the image captured by the camera, which allows for real-time detection of mask wear on the embedded device.

## III. ANALYSIS OF EXPERIMENTAL RESULTS

After the YOLOv5 model[14] has been trained, the network needs to be pruned and compressed. In this paper, we use a chunked weight pruning method, compared with structured pruning, and unstructured pruning, as shown in Table 1. The original YOLOv5 model trained on our own constructed dataset contained 84.2M weights. By applying a chunked pruning scheme, the weights were reduced to 6M, achieving a compression ratio of 14 times. After reducing the weights by 92.8%, the resulting model still maintains good accuracy, with an mAP of 85.5 and an FPS of 30. Unstructured pruning maintains a high accuracy rate at the same compression rate, but is inefficient to run. The structured pruning of the network model is very efficient, with an FPS of 34.4, but the accuracy is very low, with an mAP of 46.1. The chunked weight pruning method is able to maintain high efficiency and accuracy at the same time.

Table 1. Comparison of different pruning options

| Pruning programmes | weight | Compression rate | mAP | FPS |
|---|---|---|---|---|
| No pruning | 84.2M | 1X | 92.9 | 16.6 |
| Unstructured pruning | 6.2M | 14X | 87.1 | 20.1 |
| Structured pruning | 5.9M | 14X | 46.1 | 34.4 |
| Chunking weight pruning | 6.0M | 14X | 85.5 | 30.0 |

After the trained model is obtained, the model can be tested, Figure 1 shows a comparison of some of the experimental results.



Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 06:38:59 UTC from IEEE Xplore. Restrictions apply.



Figure 1. Comparison of experimental results

Ultimately it is the YOLOv5 model[15] that is to be compressed and then deployed onto the embedded platform. Figure 2 shows a schematic representation of the detection in an embedded platform.




Figure 2. Schematic representation of the detection in the embedded platform.

The embedded software system consists of an image acquisition program, an image processing program, a mask wear detection program and a display program. The image data is captured by the camera on the embedded system, processed and used as input to the mask wear detection system to detect whether a mask is being worn. Real-time detection at 25 frames per second is achieved on our embedded platform, with fast detection speed and high accuracy to meet practical application requirements.

## IV. CONCLUSIONS

In this paper, a deep learning target detection algorithm is deployed on an embedded Linux system based on practical applications to achieve real-time detection of whether people are wearing masks or not. Based on the current epidemic situation and current target detection technologies, and taking into account the needs of the mask wearing detection system on embedded devices, YOLOv5 was identified as the algorithm model for this mask wearing detection. The design and construction of hardware and software on the embedded device was carried out. The construction of the embedded Linux environment, such as the installation of the cross-compiler, the porting of U-Boot, the porting of the Linux kernel, and finally the making and porting of the root file system were carried out. The data set was acquired, created, the model was trained and the PC-based mask wear detection was implemented, followed by the compression of the model and the conversion of the model format. Finally, the algorithm model is deployed to the embedded device, thus completing the implementation of the mask wearing detection system on the embedded side. This paper proposes a chunked weight pruning model compression method, which improves detection efficiency while maintaining detection accuracy, with ideal results to meet practical application requirements..

## REFERENCES


[1] Shynk J J. Performance surfaces of a single-layer perceptron[J]. IEEE Transactions on Neural Networks, 1990, 1(3): 268-274.

[2] Rumelhart D E. Hinton G E. Williams R J. Learning internal representations by error propagalion[J]. Nature. 323. 1986

[3] Girshick R, Donahue J, Darrell T, et al. Rich feature hierarchies for accurate object detectionand semantic segmentation[C]. // IEEE Conference on Computer Vision and PatternRecognition, Columbus, USA, June 23-28, 2014. IEEE Computer Society, 2014. 580-587.

[4] Jack L B, Nandi A K. Support vector machines for detection and characterization of rolling element bearing faults[J]. Proceedings of the Institution of Mechanical Engineers, Part C: Journal of Mechanical Engineering Science, 2001, 215(9): 1065-1074.

[5] He K , Zhang X , Ren S , et al. Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition[J]. Pattern Analysis & Machine Intelligence IEEE Transactions on, 2015,37(9):1904-1916.

[6] Li J , Liang X , Shen S , et al. Scale-Aware Fast R-CNN for Pedestrian Detection[J]. IEEE Transactions on Multimedia, 2017.

[7] Sermanet P, Eigen D, Zhang X, et al. OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks[J]. Eprint Arxiv, 2013.

[8] Singh B , Li H , Sharma A , et al. R-FCN-3000 at 30fps: Decoupling Detection and Classification[J]. 2017.

[9] Liu W, Aanguelov D, Erhan D, et al. SSD: Single shot multibox detector[C]. // Proceedings of the 14th European Conference on Computer Vision, Amsterdam, The Netherlands, October 11-14, 2016. Lecture Notes in Computer Science 9905, Springer 2016. 21-37.

[10] Redmon J, Divvala S, Girshick R, et al. You only look once: Unified, real-time object detection[C]. // IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas,USA, June 27-30, 2016. IEEE Computer Society, 2016. 779-788.

[11] G. E. Hinton,R. R. Salakhutdinov. Reducing the Dimensionality of Data with Neural Networks[J]. Science,2006,313(5786).

[12] Liu L, Ouyang W, Wang X, et al. Deep Learning for Generic Object Detection: A Survey[J].2020.

[13] WANG C Y, LIAO H Y M, WU Y H, et al. CSPNet: A new backbone that can enhance learning capability of CNN. CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW). Seattle:IEEE,2020:1571–1580.

[14] Redmon J, Divvala S, Girshick R, et al. You Only Look Once:Unified, Real-Time Object Detection[C]//2016 IEEE Conference on Computer Vision and Pattern Recognition. 2016: 779-788.

[15] Cai Y , Li H , Yuan G , et al. YOLObile: Real-Time Object Detection on Mobile Devices via Compression-Compilation Co-Design[J]. 2020.




Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 06:38:59 UTC from IEEE Xplore. Restrictions apply.